

Government Savings

Coverage-Delta Gate — Technical Implementation Plan

Prepared by: Swapnil Patil
QA Automation & Quality Program Lead, Government Savings

Assessment Date: July 21, 2026

Version: 1.0

Classification: Confidential — Internal Use

Code Coverage Gate Implementation Plan

Code Coverage Gate — Technical Implementation Plan

****As of:**** 2026-07-21

****Scope:**** Government Savings application repositories (UniteMSC primary)

****Principle:**** Phased, repository-specific; retain existing merge controls

1. Objectives

1. Measure ****application source-code coverage (metric A)**** on every MR/PR.
2. Prevent ****material regression**** versus `main` and enforce ****changed-code**** thresholds.
3. Publish a ****required MR status check**** without weakening existing Snyk, approval, or discussion rules.
4. Provide ****auditable exceptions**** via a restricted bypass group.
5. Keep ****business automation coverage (metric B)**** in the separate coverage register.

2. Current baseline (revalidated)

| Finding | Evidence | Status |

|-----|-----|-----|

| JaCoCo present on 13 UniteMSC Java services | `pom.xml` scans | Verified from repository code |

| SonarQube disabled | `RUN\_SONARQUBE: false` | Verified from repository code |

| Static `jacoco-check` thresholds vary 0%-90% | POM grep | Verified from repository code |

| No `coverage:` regex or `coverage\_report` in QA CI YAML | `.gitlab-ci.yml` | Verified absent |

| No branch-vs-main comparison script | Repo scan | Verified absent |

| GitLab protected-branch settings | Org standard | Pending live verification (API 401) |

3. Recommended gate design

Selected model: Option 2 + Option 3 (phased)

| Check | Phase 1 | Phase 2 | Phase 3 |

```
|-----|-----|-----|-----|
| Unit/integration tests on MR | Yes | Yes | Yes |
| JaCoCo XML artifact | Yes | Yes | Yes |
| Changed-code coverage | Report only | Warn | Fail |
| Overall vs `main` regression | Report only | Fail >2pp drop | Fail >1pp drop |
| MR status check | Optional | Recommended | **Required** |
| Merge block | No | Soft (override) | Yes |
| Exception audit | N/A | Pilot | Monthly register |
**Repository-specific thresholds (examples):**
| Repository | Changed-code min | Regression tolerance |
|-----|-----|-----|
| `unite-mobile2` | 80% lines | 1pp overall |
| `unite-profile` | 80% lines | 1pp overall |
| `unite-account` | 50% lines | 2pp overall |
| Legacy 0% services | 60% changed-code only | 0pp drop first 90 days |
```

4. Architecture

...

MR opened

- CI: mvn verify (existing test stage)
- JaCoCo: target/site/jacoco/jacoco.xml
- Fetch main baseline artifact (last green main pipeline)
- Python/GitLab CI script: compare + changed-code diff
- Publish GitLab coverage + MR note + pass/fail job
- If fail: block merge (Phase 3) unless exception group approves
- Log exception to coverage-exception-register.csv

****Baseline storage:**** GitLab job artifact from `main` (30-day retention minimum) or object storage via DevOps template.

5. Implementation phases

Phase 1 — Informational (days 0–30)

| # | Task | Owner | Deliverable |

|---|-----|-----|-----|

| 1.1 | Select pilot repo (`unite-mobile2`) | QA + DevOps | Decision recorded |

| 1.2 | Add JaCoCo XML artifact to MR pipeline | DevOps | Pipeline MR |

| 1.3 | Implement `compare\_jacoco.py` (read-only) | QA Automation | Script in KB tools |

| 1.4 | MR comment with delta summary | QA Automation | Sample MR output |

| 1.5 | Document thresholds in decision brief | QA Lead | Approved thresholds |

Phase 2 — Soft gate (days 31–60)

| # | Task | Owner | Deliverable |

|---|-----|-----|-----|

| 2.1 | Fail job on threshold breach | DevOps | CI job `coverage-delta-check` |

| 2.2 | Configure exception group in GitLab | DevOps | Group + approval rule |

| 2.3 | Exception form fields (reason, expiry) | QA Governance | Template |

| 2.4 | Extend to `unite-profile`, `unite-account` | DevOps | 2 additional repos |

| 2.5 | Enable Sonar pilot (optional) | DevOps | `RUN\_SONARQUBE: true` on 1 service |

Phase 3 — Required merge check (days 61–90)

| # | Task | Owner | Deliverable |

|---|-----|-----|-----|

| 3.1 | Add `coverage-delta-check` to protected-branch required statuses | DevOps | Settings change |

| 3.2 | Rollout remaining UniteMSC services | DevOps | Phased by risk tier |

| 3.3 | Monthly exception register report | QA Automation | Leadership CSV |

| 3.4 | Reconcile `unite-financial` POM inconsistency | Engineering | Fixed jacoco-check |

Phase 4 — Monitoring (ongoing)

- Trend dashboard in coverage register
- Quarterly threshold review
- Contradiction detection vs Sonar (if enabled)

6. Tooling

| Component | Tool | Notes |

|-----|-----|-----|

| Coverage generation | JaCoCo Maven plugin | Already in POMs |

| Delta comparison | Python (`xml.etree`, `diff`) | Extend `live\_validation\_build.py` pattern |

| MR integration | GitLab CI + `glab`/`curl` API | Requires valid PAT |

| Changed-code scope | `git diff origin/main...HEAD` | Standard |

| Reporting | CSV register + leadership MD | No new web app |

| Optional | SonarQube changed-code | After `RUN\_SONARQUBE` policy |

7. What QA Automation delivers directly

- `compare\_jacoco.py` and register integration
- Pilot validation on sample MRs
- Repository matrix maintenance
- Leadership reports and exception register
- Traceability standards for QA repos (metric B, separate track)

8. What requires DevOps / Engineering

- Shared pipeline template in `ascensus-gs/shared/gitlab/templates`
- Protected-branch required status configuration
- Baseline artifact retention policy

- Bypass group membership and GitLab approval rules
- Sonar enablement and credentials
- Production secrets for CI

9. Success criteria

| Criterion | Target |

|-----|-----|

- | Pilot repo MR shows coverage delta | By day 30 |
- | Documented exception with audit trail | By day 45 |
- | Required check on pilot repo | By day 90 |
- | Zero conflation of metrics A and B in leadership reports | Ongoing |
- | GitLab API connectivity | Unblocked |

10. Out of scope (initial 90 days)

- Single GS-wide coverage percentage
- Blocking merge on full business regression suite
- qTest/Jira writes from automation
- Custom web application
- Identical threshold for all 14 services

*Related: `repository-code-coverage-gate-matrix.csv`, `code-coverage-gate-decision-brief.md`,
`coverage-intelligence-assessment/08-solution/jira-qtest-ci-harmonization-plan.md`*